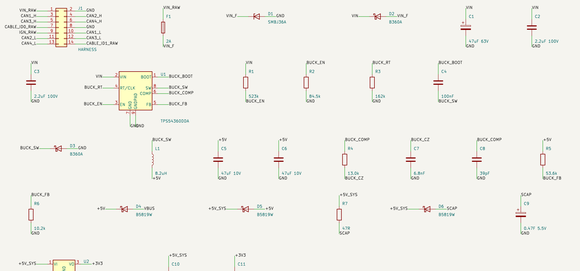
POWER: 9–36V -> fuse -> TVS -> reverse diode -> TPS54360 5V buck -> hold-up -> 3.3V LDO



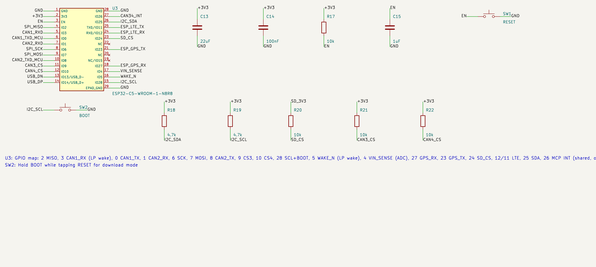
J1: Harness: 1 VIN, 2/8 GND, 3–6 CAN1–4 H, 10–13 CAN1–4 L, 9 IGN, 7/14 cable ID
D1: TVS: load dump / jump start; reverse polarity blows F1 via D1
D2: Reverse polarity block
U1: Values from TI TPS54360 datasheet 5V design example (Fig. 34)
R2: UVLO: start 8V, stop 6.25V
R3: fsw ~600 kHz
D3: Catch diode
D4: USB can power the whole board for bench work
D5: Isolates hold-up rail from buck/transceivers
R7: Supercap charge limit (~100 mA)
Q6: Supercap discharge path
C9: ~4 s hold-up at 0.5 W; lets firmware close log files on power loss

VIN / IGNITION / CABLE-ID SENSE



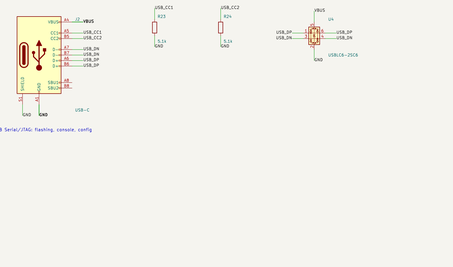
R9: VIN/11 -> ADC: 36V = 3.27V
Q1: IGN > ~6.5V pulls IGN_N low (6–60V safe)
D40: WAKE_N goes low on CAN3/CAN4 bus activity (transceiver in standby) or ignition on
R67: Deep-sleep wake sources: GPIO3 CAN1_RXD, GPIO1 CAN2_RXD, GPIO9 WAKE_N
R16: Cable grounds IDO/ID1 to identify them! (1M 3 uA in sleep)

MCU: ESP32-C5-WROOM-1 (2x native CAN-FD, Wi-Fi 6 + Wi-Fi Aware)



U3: GPIO map: 2 MISO, 3 CAN1_RX (LP wake), 0 CAN1_TX, 1 CAN2_RX, 6 SCK, 7 MOSI, 8 CAN2_TX, 9 CS3, 10 CS4, 28 SCL•BOOT, 5 WAKE_N (LP wake), 4 VIN_SENSE (ADC), 27 GPS_RX, 23 GPS_TX, 24 SD_CS, 12/11 LTE, 25 SDA, 26 MCP INT (shared, open-drain)
SW2: Hold BOOT while tapping RESET for download mode

USB-C (native USB Serial/JTAG)



J2: native USB Serial/JTAG: flashing, console, config

CAN1 (native CAN-FD 0) — OBD gateway pins 6/14

U5: STB has internal pull-up: transceiver boots in standby
JP1: FIT ONLY when the logger is a bus end.
JP5: TX ENABLE: no jumper = channel physically cannot transmit

CAN2 (native CAN-FD 1)

U6: STB has internal pull-up: transceiver boots in standby

CAN3 (MCP2518FD #1) + shared 40MHz clock

U7: STB has internal pull-up: transceiver boots in standby
U9: INT0/XSTBY drives transceiver STB; INT is open-drain, shared by both controllers
Y1: One clock for both MCP2518FD; expander can stop it in sleep

CAN4 (MCP2518FD #2)

U8: STB has internal pull-up: transceiver boots in standby
U10: INT0/XSTBY drives transceiver STB; INT is open-drain, shared by both controllers

microSD (SPI)

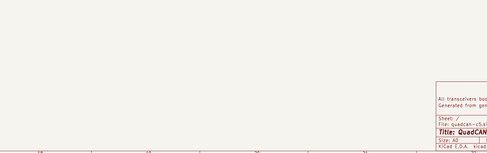
J3: SPI mode, shares the bus with the MCP2518FDs
Q5: SD card power switch: card is unpowered in sleep (idle cards can draw ~1 mA)
R65: Keep the SPI bus defined while the MCU sleeps

IO EXPANDER, STATUS LEDS, MARK BUTTON

U11: Powers up as inputs: STB lines float high = transceivers in standby
SW3: Drops an event marker into the log

RTC + CR1220 backup

GPS (optional) NEO-M9N 25Hz + active antenna bias-T



U13: Optional: leave U13 + bias-T unpopulated to rely on the phone's GPS
Q2: GPS power switch
R83: Bias-T per NEO-M9N integration manual; limits short-circuit current
R84: Blinks once per second with a fix

EXPANSION: LTE header, Qwiic

J5: LTE Cat-1 add-on (e.g. SIMCom A7670 carrier with its own regulator, fed from VIN)

MECHANICAL

FLAGS

All transceivers boot in STANDBY; TX needs firmware + a TX_EN jumper
Generated from gen/design.py — edit that, then run gen/build.sh

Sheet: /
File: quadcan-c5.kicad_sch

Title: QuadCAN-C5: 4-channel CAN-FD logger

Size: A0	Date: 2026-11-12	Rev: 1.0
KiCad E.D.A. kicad-cli 7.0.11+dfsg-1build4		Id: 1/1